

Omega Walls: A Stateful Trust Boundary for RAG and Tool-Using Agents

Anton Fedotov
SynqraTech
anvifedotov.biz@gmail.com
ORCID: 0009-0004-2313-6530

February 24, 2026

Abstract

Retrieval-augmented generation (RAG) and agentic systems routinely ingest untrusted text from external sources such as web pages, documents, emails, and tickets. Adversaries can embed directives in this content to override instruction hierarchies, exfiltrate secrets, trigger unsafe tool calls, or bypass safety constraints—turning evidence into control. We propose Omega Walls, a stateful risk runtime that sits at the trust boundary between retrieval and the LLM/tool loop. The method projects each content item through a projector π into a nonnegative wall-pressure vector, maps aggregated pressure into a saturating toxicity signal, models cross-wall “cocktail” reinforcement, and accumulates risk over time as scar-mass. A deterministic terminator predicate **Off** (with reason flags) triggers enforceable actions such as selective document blocking, tool freezing, and escalation, while attribution isolates the dominant contributing sources. We also describe an evaluation protocol (planned) for assessing attack success rate, false positives, time-to-mitigation, attribution quality, and enforcement correctness without making performance claims in this work.

1 Introduction

Prompt injection in RAG and tool-using agents is not merely “toxic text”; it is a failure of the instruction trust boundary in which untrusted content begins to behave like executable control [UK National Cyber Security Centre (NCSC), 2025, OWASP Foundation, 2024]. In practice, agents read and act on retrieved passages, attachments, and tool outputs that may contain embedded imperatives designed to hijack behavior, leak confidential data, or coerce external actions.

Common defenses—keyword filters, prompt wrapping, role separation by formatting, and post-hoc moderation—help in narrow cases but are insufficient in realistic deployments [UK National Cyber Security Centre (NCSC), 2025, OWASP Foundation, 2024]. First, attacks can be distributed across multiple retrieved chunks and across multiple agent steps, where each individual item appears innocuous but the combined effect is harmful [Liu et al., 2023]. Second, real incidents often involve

cocktails of threats (e.g., instruction override combined with tool invocation and exfiltration intent), which amplifies risk beyond what independent checks capture. Third, purely text-level defenses typically lack enforcement: even if risk is detected, the system may still execute tool calls if the LLM is partially compromised by context [Yao et al., 2022, Schick et al., 2023].

Our goal is to provide a trust-layer that does not rely on the model “behaving” in response to textual policies. Instead, we introduce a stateful mechanism that (i) treats untrusted inputs as measurable pressures on a structured threat space, (ii) accumulates these pressures over time to detect distributed attacks, and (iii) produces deterministic, auditable decisions that directly control what enters context and whether tools are permitted to execute under risk.

Conceptually, we shift from treating prompt injection as a static text-pattern detection problem to treating it as a stateful risk process at the trust boundary. In Omega Walls, untrusted inputs drive a persistent risk state, and safety becomes an auditable state machine that gates document admission and tool execution.

2 Threat Model and Design Goals

2.1 Threats

We consider RAG and agentic systems that ingest untrusted inputs from external or weakly-controlled channels, including web pages, emails, PDFs, tickets/issue trackers, chats, and tool outputs that contain third-party text. These inputs are treated as evidence to assist generation, but in practice they may also carry adversarial directives.

We focus on adversaries who embed instructions inside untrusted content with the goal of influencing the agent’s behavior at runtime. Concretely, we model four common attack objectives:

1. **Instruction override (takeover):** attempts to change instruction priority (e.g., “ignore previous instructions”, role injection, “system/developer” spoofing).
2. **Secret exfiltration (disclosure):** attempts to elicit confidential information such as tokens, passwords, API keys, system prompts, credentials, or internal data.
3. **Tool/action abuse:** attempts to coerce external actions (tool calls, commands, network requests, file writes), especially when coupled with override or exfiltration intent.
4. **Policy evasion (jailbreak):** attempts to bypass safety constraints or disable refusal behavior (e.g., “no restrictions”, “ignore safety”, “jailbreak”).

A single content item may target multiple objectives simultaneously, and attacks may be distributed across multiple retrieved chunks and/or multiple agent steps.

2.2 Design Goals (Requirements)

Our design is driven by the following requirements, motivated by practical failures of stateless, text-only defenses:

Monotonicity (non-compensation). Benign text must not “cancel” malicious pressure. Adding additional content should not reduce measured risk, ensuring that safety does not depend on adversarial mixtures of good and bad instructions.

Statefulness (accumulation over time and sources). The mechanism must accumulate weak signals across steps and across sources to detect distributed and slow-burn attacks that evade per-item or per-packet filters.

Determinism and auditability. Security decisions must be reproducible and explainable via structured outputs (e.g., reason flags, snapshots, attribution), enabling incident review, regression testing, and parameter tuning.

Selective mitigation. When risk is detected, the system should prefer targeted interventions (e.g., blocking one or two dominant documents/sources) over blunt refusal, preserving utility whenever possible.

Enforcement. Mitigations must be enforceable at system boundaries. In particular, tool execution must be physically gated (e.g., via a centralized tool gateway) so that unsafe actions are prevented even if the LLM is partially influenced by untrusted context.

3 Method: Omega Walls Formulation

Omega Walls formalizes untrusted content as measurable pressure on a structured threat space, accumulates this pressure over time, and maps the resulting state into deterministic, enforceable actions. We describe the formulation in seven parts: the wall-space taxonomy, a projector from text to wall pressures, packet aggregation, toxicity mapping with cross-wall synergy, scar-mass memory dynamics, a deterministic terminator predicate `Off`, and per-item attribution.

This reframing emphasizes runtime governance rather than content moderation: risk is represented as measurable state coupled to enforceable controls at the trust boundary (context admission and tool gating).

This reframing emphasizes runtime governance rather than content moderation: risk is represented as measurable state coupled to enforceable controls at the trust boundary (context admission and

tool gating).

Mini-glossary and notation (quick reference)

Term / symbol	Meaning (in this paper)
Ω -wall / wall	Threat cluster (risk category) in the space $\Omega = \bigsqcup_{k=1}^K \Omega^{(k)}$.
K	Number of walls (v1 uses $K = 4$).
Projector π	Maps an item x to wall pressures: $\pi : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}^K$.
x / item	Untrusted content unit (retrieved chunk, attachment, tool output, etc.).
$v(x)$ / wall pressure	Nonnegative vector; component $v^{(k)}(x)$ measures pressure on wall k .
Packet X_t	Set/sequence of items processed at step t .
v_t	Aggregated packet pressure (after combining $\{v_{t,j}\}$).
“Toxicity” p_t	Saturating risk signal derived from v_t (used for gating and deposits; not a general content-toxicity score).
Cocktail synergy	Cross-wall reinforcement: combined activation of multiple walls can amplify risk beyond independent checks.
Deposit e_t	Instantaneous deposited risk derived from v_t and p_t (feeds memory).
Scar-mass m_t	Stateful risk memory (accumulates deposits over time with decay), used to detect distributed/slow-burn attacks.
Off	Deterministic terminator predicate (with reason flags) that triggers enforceable mitigations.
Attribution / top_docs	Identifies dominant contributing items to enable selective blocking/quarantine.

Table 1: Mini-glossary and notation used throughout the Omega Walls formulation.

3.1 Ω -space: wall taxonomy

Let K denote the number of walls. In **v1** we use $K = 4$ walls:

$$\Omega = \bigsqcup_{k=1}^K \Omega^{(k)} \quad (\text{disjoint union}).$$

Each wall corresponds to a distinct class of “impossibilities” (risk clusters):

1. $\Omega^{(1)}$: **Override / Instruction Takeover**
2. $\Omega^{(2)}$: **Secret Exfiltration / Disclosure**
3. $\Omega^{(3)}$: **Tool / Action Abuse**
4. $\Omega^{(4)}$: **Policy Evasion / Jailbreak**

A single content item may pressure multiple walls simultaneously (e.g., “ignore policies” + “send secrets via a tool”).

$\Omega^{(1)}$ Override “ignore rules”	$\Omega^{(2)}$ Exfiltration “show token”
$\Omega^{(3)}$ Tool abuse “run / call tool”	$\Omega^{(4)}$ Policy evasion “bypass safety”

One item can activate multiple walls (cocktail attacks).

Figure 1: The Ω -wall space (v1, $K = 4$). Each wall represents a threat cluster. A single document/chunk can contribute pressure to multiple walls.

3.2 Projector π : text $\rightarrow$ wall pressure (with evidence)

Let $\mathcal{X}$ be the space of content items (retrieved chunks, attachments, tool outputs, etc.). The projector is a function

$$\pi : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}^K, \quad \pi(x) = v(x),$$

where $v(x) = (v^{(1)}(x), \dots, v^{(K)}(x))$ is the nonnegative wall-pressure vector.

Evidence channel. In addition to $v(x)$, the projector emits a structured **evidence** object (e.g., matched spans, polarity/intent, raw sub-scores). Evidence is essential for debugging, auditability, and downstream policy decisions (e.g., which sources to quarantine).

Polarity gating (intent awareness). A key design requirement is to avoid accumulating risk from benign security discussions (e.g., “do not reveal passwords”). We therefore define a polarity indicator per wall:

$$\text{pol}_k(x) \in \{-1, 0, +1\},$$

where $\text{pol}_k(x) = +1$ denotes a directive intent toward wall k (attack-like), $\text{pol}_k(x) = -1$ denotes protective/prohibitive intent, and $\text{pol}_k(x) = 0$ denotes neutral discussion. The v1 “hard gate” is:

$$v^{(k)}(x) = \max(0, \text{score}_k(x)) \cdot \mathbf{1}\{\text{pol}_k(x) = +1\}.$$

This gate is the main mechanism preventing false positives on policy/security documents.

Rule-based π_0 vs. learned π_θ . We provide a rule-based reference projector π_0 (dictionaries, action-object windows, negation handling, obfuscation-aware normalization) as a baseline implementation and diagnostic tool. A learned projector π_θ can replace π_0 as long as it conforms to the same interface: it must return $v(x)$ and evidence with intent/polarity and attribution-supporting signals.

3.3 Packet aggregation: non-compensating composition

At step t , retrieval returns a packet of n_t items:

$$X_t = (x_{t,1}, \dots, x_{t,n_t}) \in \mathcal{X}^{n_t}.$$

We compute per-item pressures $v_{t,j} = v(x_{t,j})$. To enforce monotonicity (no “good text cancels bad text”), we use non-compensating aggregation:

$$v_t = \bigoplus_{j=1}^{n_t} v_{t,j} \stackrel{\text{v1}}{=} \sum_{j=1}^{n_t} v_{t,j}.$$

Optionally, we apply a per-wall noise floor $\varepsilon \geq 0$:

$$\tilde{v}_t^{(k)} = v_t^{(k)} \cdot \mathbf{1}\{v_t^{(k)} \geq \varepsilon\}, \quad \tilde{v}_t = (\tilde{v}_t^{(1)}, \dots, \tilde{v}_t^{(K)}).$$

All subsequent computations use $\tilde{v}_t$ (we omit the tilde for readability when unambiguous).

3.4 Toxicity mapping and cocktail synergy

Raw pressure is mapped into a bounded toxicity (penetration) signal $p_t \in [0, 1]^K$ via a monotone saturating function. For $\alpha > 0$, define

$$\phi_\alpha(u) = 1 - e^{-\alpha u}, \quad u \geq 0,$$

applied componentwise:

$$p_t^{(0)} = \phi_\alpha(v_t) \in [0, 1]^K.$$

This captures diminishing returns: additional pressure matters most at low-to-moderate levels and saturates near 1.

Cocktail synergy. Many real incidents are combinations (override + tool + exfil), where the joint presence is riskier than independent checks would suggest. We encode cross-wall reinforcement with a nonnegative matrix $S \in \mathbb{R}_{\geq 0}^{K \times K}$ with $S_{kk} = 0$, and a scalar $\beta \geq 0$. Define:

$$(C_S(p))_k = \sum_{\ell \neq k} S_{k\ell} \min(p_k, p_\ell).$$

Final toxicity is:

$$p_t = \text{clip}_{[0,1]} \left(p_t^{(0)} + \beta C_S(p_t^{(0)}) \right).$$

Intuitively, if two walls are simultaneously active, each reinforces the other according to S , producing a kill-chain-like superadditivity.

3.5 Scar-mass memory dynamics

We transform instantaneous toxicity into a time-accumulating memory state $m_t \in \mathbb{R}_{\geq 0}^K$ (“scar-mass”). First define a per-step deposit:

$$e_t = v_t \odot p_t,$$

where $\odot$ denotes componentwise multiplication. The scar state updates as an exponential moving accumulation:

$$m_{t+1} = \lambda m_t + e_t, \quad \lambda \in (0, 1).$$

This is the core mechanism that turns stateless filtering into a session-level risk process: weak but consistent pressures add up, enabling detection of distributed attacks that evade per-item rules.

Why distributed attacks are detected. If each step contributes small deposits e_t below any instantaneous spike threshold, a stateless filter may never fire. In contrast, the recurrence above causes m_t to grow when deposits persist, eventually crossing deterministic thresholds (Section 3.6).

3.6 Deterministic terminator `Off` and reason flags

Omega converts (p_t, m_{t+1}) into a deterministic boolean predicate `Off` plus reason flags. Let $\tau \in (0, 1)$ be a toxicity spike threshold, $\Theta > 0$ a hard per-wall scar threshold, $\Sigma > 0$ a total scar threshold, and (θ, N) define a multi-wall threshold and required count.

We set $\text{Off}(m_{t+1}, p_t) = 1$ if any clause holds:

1. **Spike:** $\max_k p_t^{(k)} \geq \tau$.
2. **Hard wall breach:** $\max_k m_{t+1}^{(k)} \geq \Theta$.
3. **Total mass breach:** $\sum_{k=1}^K m_{t+1}^{(k)} \geq \Sigma$.
4. **Multi-wall breach:** $\#\{k : m_{t+1}^{(k)} \geq \theta\} \geq N$.

Importantly, `Off` does not imply a hard stop of the system. Instead, it triggers controlled degradation policies: selectively remove high-risk documents, freeze tools, quarantine sources, or escalate to a human depending on which walls fired and with what confidence.

3.7 Attribution: which document caused `Off`

To enable selective mitigation, Omega attributes risk to items in the packet. Using shared packet toxicity p_t , define per-item deposits:

$$e_{t,j} = v_{t,j} \odot p_t, \quad c_{t,j} = \|e_{t,j}\|_1 = \sum_{k=1}^K e_{t,j}^{(k)}.$$

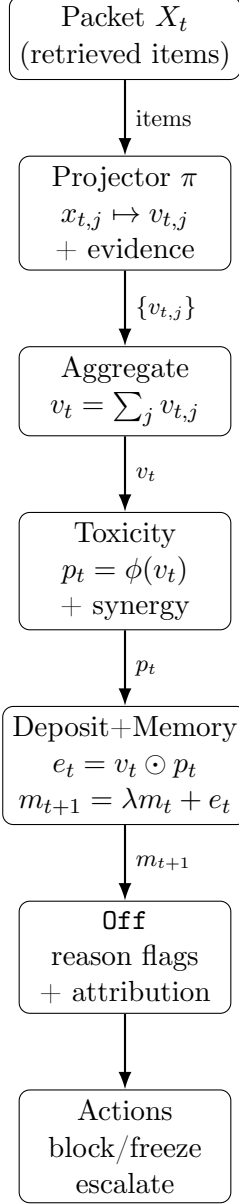


Figure 2: Computation pipeline: untrusted items are projected to wall pressures, aggregated non-compensatingly, mapped to toxicity with cocktail synergy, accumulated as scar-mass, and converted into deterministic **Off** decisions with attribution and enforceable actions.

We select the top contributor:

$$j^* = \arg \max_j c_{t,j},$$

and optionally select all items with

$$c_{t,j} \geq \gamma \cdot \max_i c_{t,i},$$

for a fixed $\gamma \in (0, 1]$ (e.g., $\gamma = 0.7$). This yields a small set of **top_docs** for targeted blocking and is also a natural basis for source-level quarantine counters.

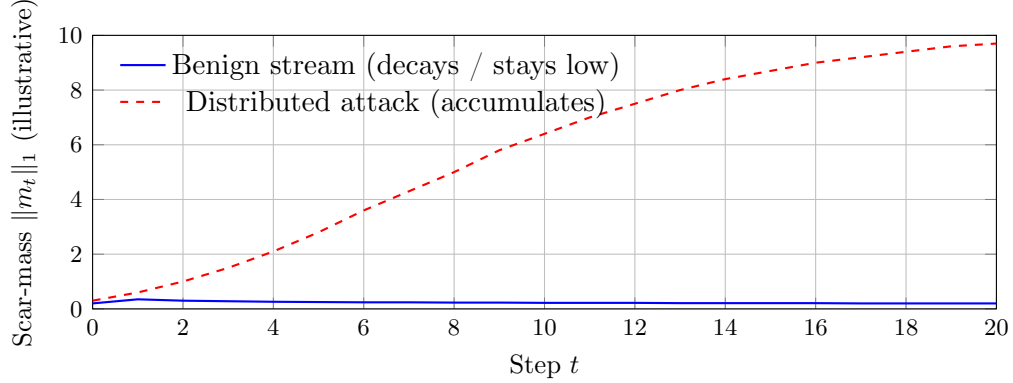


Figure 3: Illustrative scar-mass dynamics over time. With exponential decay ($\lambda < 1$), benign deposits remain low, while persistent weak pressures (distributed attacks) accumulate until deterministic thresholds can be crossed. Values are schematic; plots represent qualitative behavior of Eq. (memory update) in Section 3.5.

4 System Architecture: Where Omega Sits

Omega is designed as middleware at the trust boundary between untrusted content ingestion (retrieval, attachments, external messages, tool outputs) and the LLM/tool execution loop. Rather than relying on the model to interpret role separators correctly, Omega enforces a structured decision point: untrusted items are measured, aggregated, and either admitted as evidence or mitigated via deterministic actions.

4.1 Interfaces and data contracts

Omega integrates through three primary data contracts.

Content item. A content item is the atomic unit of projection and attribution. It includes identifiers, source metadata, and the raw text:

```
{
  "doc_id": "...",
  "source_id": "...",
  "source_type": "web|email|pdf|ticket|chat|other",
  "trust": "untrusted|semi|trusted",
  "text": "..."
}
```

Projection result. For each item, the projector returns a wall-pressure vector and structured evidence used for audit/debugging:

```
{
  "v": [ ...K floats... ],
  "evidence": {
    "polarity": [ ...K ints... ],
    "debug_scores_raw": [ ...K floats... ],
    "matches": { ... }
  }
}
```

Omega step result. Given the current scar state and the packet pressures, the Omega core emits deterministic diagnostics and mitigation signals:

```
{
  "v_total": [ ... ],
  "p": [ ... ],
  "m_next": [ ... ],
  "off": true|false,
  "reasons": [ ... ],
  "top_docs": [ ... ]
}
```

These artifacts enable reproducible enforcement, postmortems, and regression testing.

4.2 Deployment patterns

Omega supports multiple deployment modes:

1. **Library mode (in-process):** Omega runs as a library inside the agent service (lowest latency, simplest debugging).
2. **Sidecar service:** Omega runs as a local HTTP/gRPC service beside the agent (centralized logging and shared policy).
3. **Gateway/middleware:** Omega sits between the application and the LLM provider (reuse across products; requires clear contracts for retrieval packets and tool calls).

We treat these as equivalent realizations of the same trust-layer, differing only in operational concerns (latency, isolation, and centralized control).

4.3 Context builder isolation

A practical RAG/agent prompt must maintain a strict separation between trusted instructions and untrusted evidence. We recommend that the context builder compose the model input from:

- **Trusted:** system/developer instructions, user request, and internal policy directives.
- **Untrusted:** retrieved chunks and attachments, included only as evidence.

Omega directly controls what enters the evidence section: items selected by `SOFT_BLOCK` (or source quarantine) are excluded from the context, while admitted items are clearly labeled as untrusted evidence.

4.4 Tool Gateway: the enforcement chokepoint

Tool usage is the highest-leverage failure mode for injected instructions: once the agent can execute external actions (network, file writes, messaging, purchases), a single successful injection can cause real-world harm. Therefore, Omega assumes a single chokepoint for tool execution: a Tool Gateway through which all tool calls must pass.

Omega-driven actions such as `TOOL_FREEZE` and allowlists provide enforcement that is independent of the model’s “intent” or compliance with textual constraints. When the freeze mode is active, tool execution is blocked regardless of what the LLM outputs; when an allowlist is used, only approved tools and parameter patterns can proceed.

5 Reaction Policy: From Mathematics to Actions

Omega’s output is not a single “risk score” but a set of deterministic artifacts designed for enforcement and audit: wall pressures and toxicity snapshots, scar-mass updates, a boolean `off` decision, reason flags indicating which threshold clauses fired, and attribution signals (`top_docs`) identifying the dominant contributing items.

5.1 Mitigation ladder

We treat mitigation as a controlled degradation ladder, escalating only as needed:

1. **Block documents (DOC):** selectively exclude attributed items from the context (e.g., `SOFT_BLOCK` of `top_docs`).
2. **Quarantine sources (SOURCE, optional):** if repeated triggers originate from the same source, temporarily quarantine that source to prevent further retrieval.
3. **Freeze tools (TOOLS):** restrict or disable tool execution (`TOOL_FREEZE`), optionally via allowlists.
4. **Escalate (AGENT/HUMAN):** emit an incident packet and request human review when high-severity walls (e.g., exfiltration) participate or when repeated triggers occur.

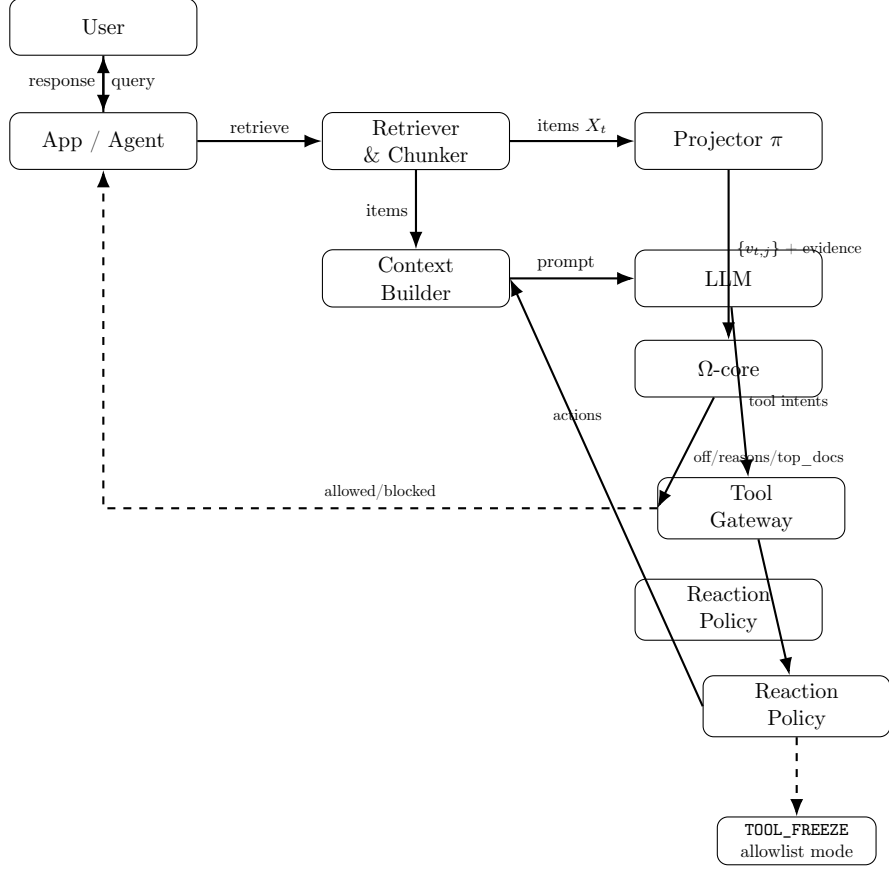


Figure 4: Simplified end-to-end flow for RAG+Omega+Tools. Omega sits between retrieval and context construction, and enforces tool safety via a centralized Tool Gateway. Reaction policy maps deterministic Omega artifacts to actions such as document blocking, tool freezing, and escalation.

5.2 Separation of concerns

The reaction policy is deliberately separated from the Omega core. Omega provides measurable, deterministic signals; the policy layer encodes an organization’s risk appetite (e.g., thresholds for escalation, tool allowlists, quarantine horizons) and can be adapted without changing the mathematical core. This separation enables consistent auditing while supporting deployment-specific operational requirements.

6 Observability and Audit

A core requirement for a trust-layer is reproducibility: given the same inputs and parameters, the system should reach the same decision and provide enough evidence to explain and replay it. Omega therefore emits structured, machine-readable events at runtime, enabling incident response, regression testing, and parameter tuning.

6.1 Structured events

We define two primary event families.

omega_off_v1 (decision event). Emitted whenever **Off** fires. This event contains:

- identifiers: session/step, relevant doc/source ids,
- snapshots: v_t , p_t , m_{t+1} ,
- reason flags: which **Off** clauses fired (spike / hard-wall / sum / multi-wall),
- attribution: **top_docs** (and optionally per-doc contributions),
- evidence summaries from the projector(s).

The decision event provides a compact incident packet that supports postmortems and reproducible replay.

enforcement_step_v1 (enforcement state event). Emitted at each step to record which enforcement controls are active. This event includes:

- tool freeze state (mode, allowlist, remaining horizon),
- source quarantine state (which sources are quarantined, horizons),
- active actions (e.g., **SOFT_BLOCK**, **TOOL_FREEZE**, **HUMAN_ESCALATE**).

This continuous event stream makes enforcement auditable: it allows operators to confirm that mitigations were actually applied, not merely recommended.

6.2 Why audit matters

These events serve three concrete purposes:

1. **Incidents:** triage and root-cause analysis (which source/doc triggered, which wall(s), and what action prevented harm).
2. **Regression testing:** replay past incidents and ensure changes to π , thresholds, or tooling do not reintroduce vulnerabilities or increase false positives.
3. **Tuning and governance:** calibrate parameters (e.g., $\varepsilon, \lambda, \tau$) and policy ladders using real traffic distributions and organizational risk appetite.

7 Validation and Results

Validation was performed locally (no external API keys) on the project version dated February 19, 2026. We used three levels of checks:

1. regression tests (`python -m pytest -q`),
2. an end-to-end quick suite (`python -m omega eval --suite quick --strict`),
3. an extended projector evaluation loop (`run_eval.py`).

7.1 Regression tests

All regression tests passed: **35/35 passed**.

7.2 End-to-end quick suite

In the quick suite, the system exhibited stable end-to-end behavior:

- `attack_pass_rate` = 1.0,
- `benign_pass_rate` = 1.0,
- `FP` = 0, `FN` = 0,
- `freeze_enforcement_rate` = 1.0,
- steps until
textttOff: `attack` = 1, `distributed_attack` = 5,
- `hard_negative_fp_rows` = 0,
- `canonical_positive_hit_rate` = 1.0.

In this demonstration loop, attacks are detected and blocked deterministically, benign scenarios do not degrade, and `TOOL_FREEZE` effectively prevents tool execution when protection is active.

7.3 Extended evaluation loop

In the extended evaluation loop (`run_eval.py`), we observed:

- hard negatives: **50 samples**, `FP` = 0,

- canonical positives: **40 samples, overall hit** = 1.0 (hit = 1.0 across all four walls),
- generator suite: **700 samples, overall hit** = 0.9326, with per-wall hit rates:
 - override: **0.8559**,
 - secret exfiltration: **0.8925**,
 - tool abuse: **1.0**,
 - policy evasion: **1.0**,
 - multi-hit rate: **0.93**.

Separately, a white-box bypass block showed high bypass in the current baseline (**bypass_rate** = 0.8108 over **40** evaluated samples). We treat this explicitly as a target for the next iteration, consistent with positioning the repository as a reference risk runtime and evaluation harness rather than a finalized enterprise security platform.

8 Planned Evaluation Protocol

We also outline a general evaluation protocol intended to support reproducible, deployment-relevant measurements and to guide future extensions.

8.1 Metrics

We propose the following metrics for evaluating a trust-layer in RAG and tool-using agents:

- **Attack Success Rate (ASR):** fraction of adversarial trials in which the system performs a disallowed outcome (e.g., secret disclosure or unsafe tool execution).
- **False Positive Rate (FPR):** fraction of benign trials in which the system blocks evidence, freezes tools, or escalates unnecessarily.
- **Time-to-mitigation:** number of agent steps until effective mitigation activates (e.g., **Off** fires or tools are frozen), especially under distributed attacks.
- **Attribution quality:** whether **top_docs** correctly identifies the dominant contributing document(s)/sources in labeled attack scenarios.
- **Enforcement correctness:** whether blocked documents are excluded from context and whether tool restrictions prevent execution under risk.
- **Latency overhead:** added runtime cost from projection, aggregation, and enforcement checks.

8.2 Evaluation suites

We propose suites that reflect both realistic traffic and adversarial diversity:

- **Benign hard negatives:** legitimate security documentation, compliance text, and operational runbooks that mention “secrets” and “tools” but are protective (to test polarity gating and reduce over-refusal).
- **Red-team positives:** direct and indirect prompt injection samples targeting each wall (override, exfiltration, tool abuse, evasion).
- **Obfuscations/paraphrases:** spaced letters, homoglyph substitutions, quoting, and rephrasings designed to evade naive string matching.
- **Distributed/cocktail scenarios:** attacks split across multiple retrieved chunks and/or multiple agent steps, including combinations of objectives (e.g., override + tool + exfil).

8.3 Baselines

To contextualize Omega Walls, we recommend comparisons against practical local baselines:

- **No defense:** standard RAG/agent pipeline without filtering or enforcement.
- **Keyword/regex filter:** heuristic string matching over retrieved content and/or model outputs.
- **Stateless scanner:** per-item or per-packet classifier/LLM-judge that does not accumulate session state.
- **Tool-only gateway:** strict allowlists or disabled tools without content-based risk accumulation.

8.4 Statement of scope

We emphasize that the above protocol is intended to produce reproducible, deployment-relevant measurements. Results will be reported in a subsequent release/work after completing dataset curation and calibration.

9 Related Work

Omega Walls sits at the intersection of LLM guardrails, red-teaming/scanning, tool enforcement, and enterprise runtime security. We summarize adjacent lines of work by class and clarify how our formulation combines stateful risk accumulation, deterministic termination, attribution, and enforceable actions into a single trust-layer.

9.1 Guardrails frameworks

Guardrails frameworks typically provide rule-based or model-assisted checks around prompts and outputs (e.g., input/output validators, schema constraints, toxicity filters, policy templates). These systems can be effective for structured output validation and content moderation, and can reduce certain classes of prompt injection when combined with strong prompt isolation. However, many guardrails approaches are primarily text-level controls and are often stateless with respect to the agent session: they evaluate each prompt or response largely in isolation, and may not explicitly model distributed attacks that accumulate across steps or across multiple retrieved sources. Omega Walls complements guardrails by introducing a session-scoped risk state (scar-mass) and a deterministic terminator predicate that drives enforceable mitigations.

9.2 Scanners and red-teaming tools

Red-teaming and scanning tools focus on discovering vulnerabilities rather than enforcing mitigations at runtime. They generate adversarial prompts, probe jailbreak behaviors, and measure model susceptibility under different prompting conditions. Such tools are valuable for evaluation and regression testing, but are not typically designed as a runtime trust boundary: they do not sit in the inference path, do not gate tool execution, and do not maintain a persistent risk state per session. In our framing, scanning is complementary: it produces adversarial suites that can be used to test projectors π and the Omega core invariants.

9.3 Tool firewalls and allowlists

Tool firewalls (e.g., strict allowlists, parameter validators, and execution proxies) provide enforcement by restricting which tools can be executed and under what constraints. This is a critical component of safe agent design. However, tool-only enforcement often lacks (i) a structured representation of content-borne threats in retrieved evidence and attachments, (ii) session-scoped accumulation that captures distributed attacks, and (iii) attribution of risk back to specific documents/sources for selective mitigation and quarantine. Omega Walls incorporates a tool gateway as the enforcement chokepoint, but also supplies the missing risk-state and attribution signals that determine when and why tools should be restricted.

9.4 Enterprise prompt shields and runtime AI security

Enterprise prompt shields and runtime AI security products commonly combine policy engines, classification models, and monitoring to reduce prompt injection and data leakage risks in production. These systems often emphasize operational integration (logging, governance workflows, incident handling, compliance), and may include both input and output controls. Omega Walls aligns with this operational goal but differs in its method-level contribution: it defines a compact, implementable mathematical core that (i) treats untrusted content as measurable wall pressures, (ii) accumulates session-scoped risk via scar-mass, (iii) triggers a deterministic terminator predi-

cate **Off** with reason flags, and (iv) supports selective mitigation through attribution. In other words, Omega Walls can be viewed as a reference risk-runtime component that can sit beneath (or alongside) higher-level enterprise control planes.

Class	Stateful	Deterministic Off	Attribution	Tool enforcement
Guardrails frameworks	○	○	○	○
Scanners / red-teaming	–	–	○	–
Tool firewalls / allowlists	–	○	○	✓
Enterprise prompt shields	○	○	○	○
Omega Walls (this work)	✓	✓	✓	✓

Table 2: High-level comparison by capability. Symbols: ✓ present as a first-class mechanism; ○ may be present in some systems but not central or not standardized; – not applicable/typically absent. Omega Walls integrates stateful risk, deterministic termination, attribution, and enforceable actions as a single trust boundary.

10 Limitations and Scope

Omega Walls is a trust-boundary mechanism for systems that ingest untrusted content and may execute tools. While the formulation is intentionally minimal and implementable, it has limitations that motivate future work.

Rule-based projector limitations. The reference projector π_0 is rule-based and therefore limited under paraphrase-heavy, semantic, or highly novel injections. While normalization and intent/polarity heuristics help against simple obfuscations, robust coverage against diverse linguistic strategies may require a learned projector π_θ or hybrid approaches (rules + model signals) that preserve the same output interface and auditability properties.

Calibration requirements. Omega introduces parameters (e.g., $\varepsilon, \alpha, \beta, \lambda$ and **Off** thresholds) that must be calibrated to a deployment’s traffic distribution, tool surface area, and risk tolerance. Conservative thresholds can increase over-blocking; permissive thresholds can delay mitigation. We therefore treat calibration and governance as a first-class operational step, supported by structured events (Section 6) and planned evaluation suites (Section 8).

Scope of protection. Omega Walls does not claim to solve all forms of unsafe behavior in LLM systems. In particular, it is not a general solution to hallucinations or to leakage modes that produce no detectable signals in the text or metadata available to the projector. The intended scope is trust-boundary security: preventing untrusted external text (retrieval, attachments, tool outputs) from functioning as instructions, and ensuring that tool execution is gated when risk is detected.

Operational trade-offs. Because Omega participates in the inference path, it introduces run-time overhead proportional to the number and size of items in the packet. Practical deployments may require batching, caching, and careful scoping of what is projected (e.g., which tool outputs are treated as untrusted). Additionally, selective mitigation policies (block/freeze/escalate) must be tuned to preserve utility under benign-but-sensitive content (e.g., compliance documents that mention “tokens” and “tools”).

11 Future Work

Omega Walls is intentionally designed as a minimal, implementable core. Several extensions can improve robustness, automation, and coverage while preserving the same audit-friendly interface.

Learned projectors and hybrid signals (π_θ). A natural next step is to replace or augment the rule-based reference projector π_0 with a learned projector π_θ that better captures paraphrases, semantic injections, and novel attack patterns. Importantly, π_θ should remain drop-in compatible: it must output a nonnegative wall-pressure vector $v(x)$ and structured evidence (including polarity/intent and supporting spans) to preserve auditability and attribution.

Automatic threshold tuning and governance. Omega introduces tunable parameters (e.g., $\varepsilon, \alpha, \beta, \lambda$ and `Off` thresholds) that trade off safety and utility. Future work includes (i) data-driven calibration procedures that adapt thresholds to deployment traffic, (ii) per-tenant or per-workflow profiles for different risk appetites, and (iii) policy-level guardrails that prevent unsafe configurations. These efforts can be supported by replayable event logs and offline simulation of mitigation ladders.

Source-scoped memory and quarantines. While the current formulation is session-scoped, many deployments benefit from source-scoped memory: tracking repeated triggers per source (domain, sender, document repository) to enable automatic quarantine, rate limiting, or trust downgrading. This adds a second axis of accumulation (session $\times$ source), enabling defense against recurring low-grade adversarial sources.

Output gating and DLP extensions. Omega currently targets inbound untrusted content and tool execution. A complementary direction is output gating: detecting and preventing sensitive data leakage (DLP) in model responses. This can be implemented as an additional projector (or a new wall) operating over generated text and tool outputs, triggering deterministic actions such as redaction, refusal, or escalation when leakage is detected.

Deeper integration with evaluation corpora and CI invariants. To ensure long-term stability, we plan tighter integration with evaluation corpora and continuous integration (CI) invariants: hard-negative suites (benign security docs), red-team positives, obfuscation sets, and

distributed/cocktail scenarios. The goal is to provide regression tests that ensure (i) benign-only streams do not trigger `Off` within a chosen horizon, and (ii) known attacks reliably trigger appropriate mitigations without regressions in enforcement correctness.

12 Open-Source Release

We release Omega Walls as an open reference implementation intended to standardize how prompt injection risk is measured and enforced in RAG and agentic systems.

Open components (OSS). The open-source release includes:

- a reference Ω -core implementing the formulation (pressure aggregation, toxicity mapping, synergy, scar-mass update, `Off`, and attribution),
- the baseline rule-based projector π_0 with evidence and polarity gating,
- a minimal evaluation harness and example suites (hard negatives, red-team positives, obfuscations, distributed/cocktail scenarios),
- lightweight adapters for common RAG pipelines (retrieval packet interfaces, chunking conventions, and context-builder isolation patterns).

Enterprise extensions. Production deployments often require operational controls beyond the reference core. Enterprise extensions typically include:

- a policy/control plane for configuring thresholds, risk ladders, allowlists, and multi-tenant governance,
- hardened Tool Gateway integration (robust parameter validation, sandboxing, and organization-wide enforcement),
- observability and compliance integration (SIEM, SSO/RBAC, audit trails, incident workflows),
- managed tuning and continuous updates informed by deployment-specific traffic and evolving attack patterns.

This separation preserves transparency and reproducibility at the method level while supporting operational requirements in production environments.

The open-source repository is available at github.com/synqrtech/omega-walls.

13 Related Work

We situate Omega Walls at the intersection of prompt-injection threat modeling, prompt-injection benchmarking, and runtime guardrails/enforcement for tool-using agents.

Threat models and security taxonomies. Industry security guidance highlights that prompt injection blurs the boundary between instructions and data, motivating explicit trust boundaries and enforceable mitigations [UK National Cyber Security Centre (NCSC), 2025, OWASP Foundation, 2024]. We also reference established AI security taxonomies such as MITRE ATLAS for mapping techniques and building consistent evaluation suites [MITRE, 2025].

Formalization and benchmarking of prompt injection. Recent work formalizes prompt-injection attacks and defenses and provides benchmarking methodology [Liu et al., 2023]. Open benchmarks and toolkits enable more systematic, reproducible evaluation beyond anecdotal red-teaming [et al., 2023].

Runtime guardrails and enforcement components. In industry, prompt-shielding and content-safety components provide deployable detection and filtering mechanisms for both direct user attacks and indirect document-driven attacks in RAG-style pipelines [Microsoft Learn, 2026, 2025, Microsoft Azure, 2025]. Open-source frameworks and platforms similarly implement validation and policy layers around inputs/outputs and tool gateways [guardrails-ai, 2025, openguardrails, 2025].

Agents and tool use. Agentic systems that interleave reasoning with tool execution amplify the importance of enforceable tool gating and stateful risk tracking [Yao et al., 2022, Schick et al., 2023].

References

- Liu et al. Open-prompt-injection: Benchmark for prompt injection attacks and defenses, 2023. URL <https://github.com/liu00222/Open-Prompt-Injection>. Code repository. Accessed 2026-02-19.
- guardrails-ai. Guardrails: Adding guardrails to large language models, 2025. URL <https://github.com/guardrails-ai/guardrails>. Open-source framework. Accessed 2026-02-19.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses, 2023. URL <https://arxiv.org/abs/2310.12815>. arXiv:2310.12815.
- Microsoft Azure. Azure ai content safety, 2025. URL <https://azure.microsoft.com/en-in/products/ai-services/ai-content-safety/>. Product overview. Accessed 2026-02-19.

- Microsoft Learn. Content filter prompt shields for documents (azure openai), 2025. URL <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/content-filter-prompt-shields>. Indirect/document attack framing. Accessed 2026-02-19.
- Microsoft Learn. Prompt shields in azure ai content safety, 2026. URL <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection>. Concepts and usage. Accessed 2026-02-19.
- MITRE. Atlas / adversarial threat landscape for ai systems (advml threat matrix), 2025. URL <https://github.com/mitre/advmlthreatmatrix>. Repository. Accessed 2026-02-19.
- openguardrails. Openguardrails: Developer-first open-source ai security platform, 2025. URL <https://github.com/openguardrails/openguardrails>. Mentions technical report arXiv:2510.19169. Accessed 2026-02-19.
- OWASP Foundation. Owasp top 10 for large language model applications (version 1.1), 2024. URL <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. LLM01: Prompt Injection. Accessed 2026-02-19.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>. arXiv:2302.04761.
- UK National Cyber Security Centre (NCSC). Prompt injection is not sql injection (it may be worse), 2025. URL <https://www.ncsc.gov.uk/pdfs/blog-post/prompt-injection-is-not-sql-injection.pdf>. PDF. Accessed 2026-02-19.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>. arXiv:2210.03629 (ICLR 2023).